

Rapport du projet Modèles et Méthodes pour les Systèmes multi-agents (M1 CNS SA)

Max Fillère

Boudjémâa Salah

Sommaire :

1. Introduction
2. Fonctionnement général
3. Implémentation dans le code
4. Difficultés rencontrées
5. Limites et améliorations
6. Conclusion
7. Annexes

1. Introduction

Pour introduire le projet, nous avons comme indication le site <https://ncase.me/trust/> pour nous introduire sur le principe du jeu de l'évolution de la confiance.

Le principe de ce jeu est simple, on a des joueurs qui peuvent soit coopérer (dire la vérité), soit trahir (mentir), pour gagner un certain nombre de pièces (coins). Dans le cas général, on a deux joueurs qui s'affrontent et en fonction du comportement de chaque joueur, il en ressortira un résultat différent. On va pouvoir par la suite en étudier le comportement et l'efficacité de chacun des comportements dans un environnement avec plusieurs joueurs.

Nous avons donc modélisé ce jeu de l'évolution de la confiance pour, dans un premier temps, avoir un code fonctionnel pour visualiser la façon dont le jeu fonctionne de manière la plus simpliste et efficace possible, puis dans un second temps, pour en analyser et comprendre les stratégies les plus optimales en fonction de l'environnement défini, par exemple à quel point un joueur qui ment tout le temps ne sera pas gagnant dans un environnement dans lequel tous ses adversaires coopèrent entre eux, ou encore à quel point le taux d'erreur impact l'évolution d'un environnement donné.

2. Fonctionnement général

Une interface nous permet de choisir le nombre de joueurs que l'on souhaite pour chacune des stratégies existantes dans notre modélisation. On définit une stratégie comme étant la manière qu'aura le personnage de jouer face à un joueur.

(mensonge ou vérité en fonction de divers paramètres)

On possède donc plusieurs **stratégies** :

- **humain** : Le choix (vérité/mensonge) est fait via l'interface par l'utilisateur
- **bot** : Les choix sont faits de manière aléatoire. L'interface possède un "slider" « **bot-truth-percent** » grâce auquel on peut choisir à combien de pourcentage le bot dira la vérité (plus ce pourcentage sera élevé, plus le bot dira la vérité).
- **copycat** : Il commence par dire la vérité et joue ensuite le dernier coup de l'adversaire
- **cheater** : Il mentira à tous les coups
- **cooperator** : Il dira la vérité à tous les coups
- **grudger** : Il commence par dire la vérité, et, à partir du premier mensonge d'un adversaire, mentira à tous ses prochains coups
- **detective** : Il commence toujours par les 4 coups suivants : vérité, mensonge, vérité, vérité, et décide ensuite de faire soit :
 - si l'adversaire n'a jamais menti : Il ment en permanence
 - si l'adversaire a menti au moins une fois : Il répète le dernier coup du joueur (comme un **copycat**)
- **simpleton** : Il commence par dire la vérité puis :
 - si son adversaire dit la vérité, il continuera à dire la vérité
 - si le dernier coup de son adversaire est un mensonge, il inverse son propre dernier coup
- **custom** : Le plus complexe et avancé en termes de possibilité, il peut :
 - avoir un nombre de premiers coups définis (*custom-initial-moves-list*)
 - répéter ces premiers coups définis (*custom-repeat-initial-moves*)
 - jouer aléatoirement (*custom-play-randomly*) avec la possibilité de choisir son pourcentage de dire la vérité (*custom-truth-percent*)
 - jouer le dernier coup de son adversaire (*custom-play-opponent-last-move*) avec la possibilité choisir combien de coup avant de copier les coups de l'adversaire (*custom-play-opponent-last-move-number*)
 - trahir à partir d'un certain nombre de trahisons totales (*custom-betray-after*)
 - trahir à partir d'un certain nombre de trahisons consécutives (*custom-consecutive-betray-after*)

Le jeu en lui-même possède plusieurs paramètres ajustables dans l'interface :

- **speed-game** : permet de faire une pause de quelques millisecondes entre chaque match allant de 0 seconde (aussi rapide que possible) jusqu'à 1 seconde de délai entre chaque match
- **error-rate** : permet de définir un taux aléatoire d'erreur à chaque match, c'est-à-dire jouant le coup opposé de ce que le joueur était supposé jouer
- **coins-verite-verite** : pour choisir le nombre de coins gagné lorsque 2 joueurs disent la vérité (2 par défaut)
- **coins-verite-mensonge** : nombre de coins perdus lorsqu'un joueur dit la vérité et que son adversaire trahit (-1 par défaut)
- **coins-mensonge-verite** : nombre de coins gagnés lorsqu'on trahit un adversaire qui dit la vérité (3 par défaut)
- **coins-mensonge-mensonge** : nombre de coins gagnés si deux adversaires mentent (0 par défaut)
- **fast-round** : permet d'exécuter le nombre de matchs par round (*round_av_evolution*) pour rendre plus rapide l'exécution de tous les matchs plutôt que de les jouer un par un (pratique pour des simulations avec de nombreux joueurs et de nombreux tours, et pratique à désactiver quand on souhaite observer le coup que fait chaque joueur lors de chaque match)

Et enfin, on possède des **boutons** pour exécuter certaines actions :

- « **setup** » : pour initialiser toute notre modélisation et ses paramètres ;
- « **Tour complet** » : pour exécuter tous les matchs de tous les joueurs les un contre les autres avant de s'arrêter
- « **Pas à pas** » : pour effectuer le match suivant dans la liste de tous les matchs restants à jouer
- « **reset** » : pour réinitialiser le nombre de joueurs de chaque stratégie à zéro
- « **go** » : pour exécuter indéfiniment tous les matchs sans jamais s'arrêter jusqu'à ce que le bouton soit à nouveau pressé pour s'arrêter
- « **Humain vérité** » : pour choisir de dire la vérité lorsque c'est le tour d'un humain de jouer
- « **Humain mentir** » : pour choisir de mentir lorsque c'est le tour d'un humain de jouer

3. Implémentation dans le code

Le code est segmenté en plusieurs sections en fonction de leur rôle dans le projet.

1. Contrôle de l'Interface (CONTRÔLE DE L'INTERFACE)

Gère les interactions utilisateur et le déroulement de la simulation :

setup : Initialise le monde, les joueurs et les paramètres globaux

go : Lance la simulation en continu

one-round-only : Exécute un tour complet puis s'arrête

next-match-step : Exécute un match à la fois (mode pas-à-pas)

human-play : Permet à un joueur humain de prendre une décision manuelle

2. Gestion des Joueurs (PLAYER)

create-all-players : Création de tous les joueurs

setup-player : Méthode pour créer un joueur avec sa stratégie, sa couleur et sa mémoire

3. Affichage des Étiquettes (LABEL)

setup-labels : Attache une étiquette à chaque joueur

update-label-position : Ajuste la position des étiquettes

set-names : Affiche, stratégie, coins et dernier coup joué

4. Évolution de la Population (ÉVOLUTION DE LA POPULATION)

evolve-population :

Élimine les pires joueurs (faible nombre de coins)

Duplique les meilleurs joueurs

Réinitialise les scores pour le prochain round

5. Système de Monnaie (COINS)

add-coins : Ajoute ou retire des coins à un joueur

6. Matchmaking (MATCHMAKING)

start-new-round : Commence un nouveau tour

play-next-match : Joue le prochain match dans la liste des matchs restants

prepare-all-matches : Génère toutes les paires de matchs possibles

play-match : Fait s'affronter deux joueurs et applique les résultats

apply-results : Applique les résultats pour mettre à jour les coins et la mémoire

get-decision : Détermine le coup d'un joueur selon sa stratégie (ex: Copycat, Detective, Custom)

apply-results : Met à jour les scores après un match

7. Utilitaires (UTILITAIRES)

number-betrayed : Compte les trahisons d'un joueur

number-number-betrayed-consecutive : Compte les trahisons consécutives d'un joueur

get-player-history : Affiche l'historique d'un joueur dans la console

add-move-to-memory : Ajoute le coup d'un joueur dans la mémoire de son adversaire

get-moves-against : Récupère l'historique des coups contre un adversaire

update-list-worst-best-players : Identifie les meilleurs/pires joueurs pour l'évolution

4. Analyses des résultats

Pour étudier l'efficacité des stratégies, nous avons mené une série de matchs tests avec différentes configurations. Les résultats mettent en lumière des dynamiques intéressantes, notamment l'impact de l'environnement sur la performance des stratégies.

Test 1 : 1 Cheater vs 4 Cooperators

Configuration :

- Rounds avant élimination : **1 tour** (même résultat indépendamment du nombre de tours)
- Taux d'erreur : **0%** (négligeable ici)
- Élimination du pire joueur et duplication du meilleur

Résultats attendus :

- Le **Cheater** exploite les **Cooperators** en mentant à chaque tour, gagnant 3 coins par match
- Les **Cooperators** perdent 1 coin par match contre le **Cheater**, mais gagnent 2 coins entre eux.

Résultats observés :

Voir annexe image1 image2 image3

Analyse :

Le **Cheater** domine à chaque tour, mais cette stratégie est non viable en environnement évolutif :

Après quelques rounds d'évolution, les **Cooperators** sont éliminés, puis les **Cheaters** se retrouvent isolés, ne pouvant plus exploiter personne, et ne gagnant plus aucun gain.

Conclusion :

Une stratégie purement exploratrice (comme Cheater) est efficace à court terme, mais non durable dans un écosystème évolutif.

Test 2 : 1 Cheater vs 4 Copycats

Configuration :

- Rounds avant élimination : **10 tours**
- Taux d'erreur : **10%**
- Élimination du pire joueur et duplication du meilleur

Résultats attendus :

- Le **Cheater** gagnent 3 coins contre les **Copycats** au premier tour puis ne gagnent plus rien de la part de ces derniers par la suite (sauf en cas d'erreur)
- Les **Copycats** perdent 1 coin par match lors du premier match contre le **Cheater** puis n'en perdent plus, et gagnent 2 coins contre les autres **Copycats**.

Résultats observés :

Voir annexe image4

Analyse :

Au premier tour, le **Cheater** est le seul à mentir, par conséquent, il est le seul à gagner des coins. Cependant, au terme des 10 premiers tours, les **Copycats** ont coopéré entre eux, le **Cheater** à certes menti à tout le monde, mais tout le monde lui a aussi menti. De cette manière, les gains sont nuls, lors des matchs impliquant le **Cheater**. Sauf en cas d'erreur, où le résultat sera de +2 pour le **Cheater** et le **Copycat** (car +3/-1 puis -1/+3), Puis les Copycats gagneront +2/+2 à chaque tour et finiront par dominer le **Cheater** et l'éliminer

Conclusion :

Avec un comportement adaptatif du dernier coup joué, le **Copycat** peut coopérer avec les autres **Copycats** qui coopèrent (aucun impact avec le taux d'erreur à 10% pour les conséquences), puis mentir contre le **Cheater** en répétant son dernier coup, c'est une stratégie efficace pour punir les menteurs.

Notons qu'un taux d'erreur de 10% n'est pas assez impactant pour changer l'issue du test.

Test 3 : 1 Bot vs 1 Cheater vs 1 CopyCat vs 1 Cooperator vs 1 Grudger vs 1 Detective vs 1 Simpleton vs 1 Copykitten

Configuration :

- Rounds avant élimination : **15 tour**
- Taux d'erreur : **0%**
- Élimination du pire joueur et duplication du meilleur

Résultats attendus :

- On s'attend à ce que les joueurs qui font au moins une trahison se retrouvent à se faire éliminer petit à petit, laissant uniquement des joueurs coopérant entre eux à la fin

Résultats observés :

Voir annexe image5 image6 image7 image8

Analyse :

Le **Bot** se fait éliminer en premier, ce qui est logique car il n'a aucune stratégie définie et gagne le moins de coins.

Le **Cheater** est le second éliminé, comme vu précédemment il ne performe pas bien dans un environnement avec des personnes coopérant entre-elles

Puis enfin le **Detective** finit par se faire éliminer puisqu'il fait un coup "mentir" à son deuxième coup, le faisant passer en termes de coins après 15 tours.

À partir de l'image 8, on se rend compte que tous les joueurs restants obtiennent le même résultat, la suite de l'exécution n'est donc pas pertinente.

Conclusion :

Les stratégies coopératives entre elles éliminent petit à petit les joueurs ne coopérant pas dans la partie. Cela peut s'expliquer par un gain plus important venant de la coopération.

5. Difficultés rencontrées

Le premier gros problème rencontré dans notre modélisation est lié à la mémoire des joueurs par rapport à chacun de leur match. Notre première modélisation sauvegardait le dernier coup joué du joueur uniquement, sans prendre en compte l'adversaire contre lequel on jouait, l'implémentation d'une mémoire personnalisée pour chaque joueur qui enregistre chacun des coups de tous ses adversaires a été difficile à mettre en place pour obtenir ce résultat assez solide qui consiste à stocker l'id du joueur puis chacun de ses coups dans une liste, de format :

mémoire du joueur0 [1[11111] 2[01000]]

Le positionnement des noms pour indiquer visuellement leur nom, leurs coins et leur dernier coup joué, par défaut le changement du label du joueur rendait quasiment illisible (cf Annexe image9) notre modélisation et il était impossible de modifier sa position, alors on a créé une "étiquette" qui permet d'afficher ces informations de façon claire. On a eu pas mal de difficulté à afficher correctement la forme de cercles lisible parce qu'en fonction de la taille "size" la position variait (cf Annexe image10) et ne plaçait pas correctement les noms.

La position des joueurs donnés par défaut ([who] of player) n'était absolument pas dans l'ordre lorsqu'on avait plusieurs joueurs d'un même type et qui causait des problèmes lors de la création de nouveaux joueurs et leur placement, ce qui faisait que les matchs ne se déroulaient pas dans l'ordre, on a donc implémenté un id-placement à chaque joueur et on a trié quand nécessaire en utilisant cette variable plutôt que les id par défaut.

6. Limites et améliorations

On a pensé à pouvoir implémenter plusieurs joueurs customisables avec chacun leurs propres variables, pour ce faire, on peut soit créer un joueur « custom-bis » par exemple pour lequel on duplique toutes nos variables, mais ce n'est clairement pas une option optimale, de l'autre, on pourrait avoir des variables locales à chaque fois qu'un joueur de type custom est créé, il faudrait de ce fait qu'on fasse un bouton « ajouter joueur custom » qui pourrait sauvegarder les réglages dans l'interface des options activées ou non, pour ensuite les enregistrés pour un joueur spécifique, puis en modifiant les réglages ça ne changerait rien au joueur déjà implémenté, et lorsqu'on clique sur un bouton « ajouter joueur custom », les nouveaux paramètres seront enregistrés avec ce nouveau joueur customisables. Le problème d'une telle solution serait qu'il ne serait pas possible d'enregistrer « définitivement » les paramètres choisis, ce qui reviendrait à chaque initialisation à

tout redéfinir de zéro ce qui ne serait vraiment pas optimal. Ou alors en implémentant avec *file-print*, *file-write*, *file-show*.

Ensuite, on est limité par le nombre de joueurs et de rounds que l'on souhaiterait exécuter, à partir de plusieurs centaines, voire milliers de joueurs, il devient beaucoup trop long de calculer et de faire s'affronter tous les joueurs les uns contre les autres, si par exemple, je voudrais modéliser 5 000 **copycats** contre 10 000 **détectives**, et que l'évolution se fasse après que chaque joueur a fait 1000 matchs contre tous les autres joueurs, on ne peut tout simplement pas modéliser cela et le temps de calcul serait probablement ridiculement élevé.

On pourrait aussi ajouter une possibilité de pardonner un joueur qu'on décide de trahir à partir d'un certain pourcentage de coopération de son adversaire ou d'une séquence de vérité consécutive.

7. Conclusion

Grâce à ce projet, on se rend compte que le comportement de n'importe quel joueur dépend de l'environnement dans lequel il se situe, la façon dont un joueur évolue dépend entièrement des joueurs qui font partie de son environnement, chaque joueur fait partie de l'environnement des autres joueurs. Ce sont les joueurs qui définissent ce jeu, le jeu ne fait que définir leur propriété, mais par la suite, ce sont les joueurs eux-mêmes qui décident en fonction des joueurs qu'ils rencontrent. On se rend compte que c'est important de réunir des conditions pour construire une relation de confiance entre les joueurs, parce qu'avec de la confiance, ça sera gagnant-gagnant à la fin, en communiquant le plus correctement possible et en acceptant que des erreurs puissent arriver, c'est la très grande partie du temps les liens de confiance qui l'emportent dans cette modélisation sur le long terme, parce que ça tire chaque joueur vers le haut, tandis que les mauvaises relations ne permettent pas de survivre dans la plupart des environnements partagés par d'autres joueurs coopératifs.

Il est important de faire en sorte dans cette modélisation qu'un résultat gagnant-gagnant soit toujours la meilleure issue possible pour tirer des conclusions dans cette simulation du jeu de l'évolution de la confiance.

Malgré tout, la bonne communication, c'est-à-dire le taux d'erreur le plus faible possible, est extrêmement important pour que les joueurs se comprennent et pardonnent les erreurs commises, pour continuer à se faire confiance par la suite. Si le taux d'erreurs est beaucoup trop élevé et que les joueurs ne comprennent plus ce que fait son environnement, ça rend certaines stratégies de confiance beaucoup plus incertaines et qui finissent souvent par ne plus faire confiance à son adversaire.

Annexe :

Image1: 1 Cheater vs 4 Cooperators (1 tour)

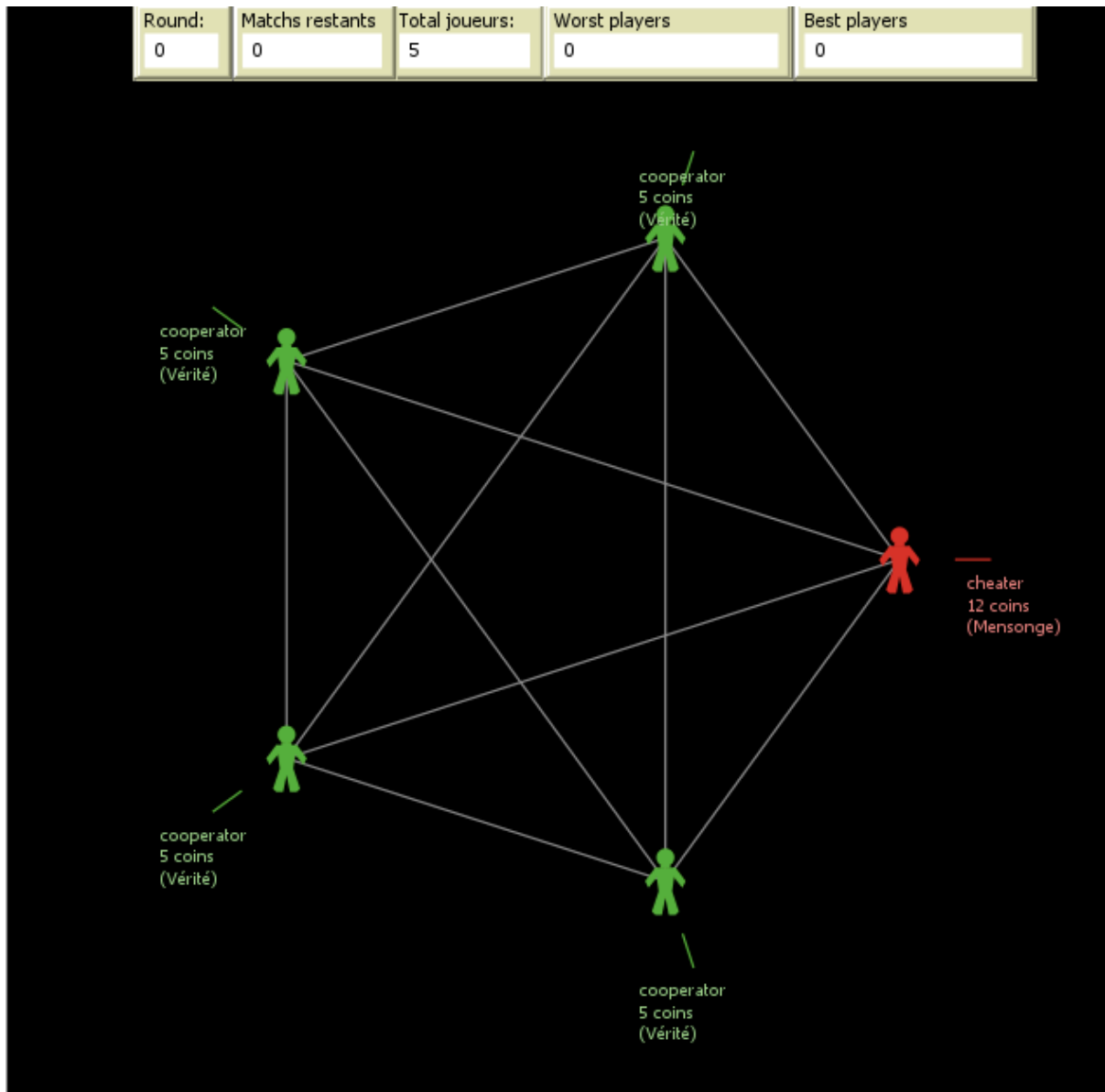


Image2: Cheater vs 4 Cooperators Tour 2

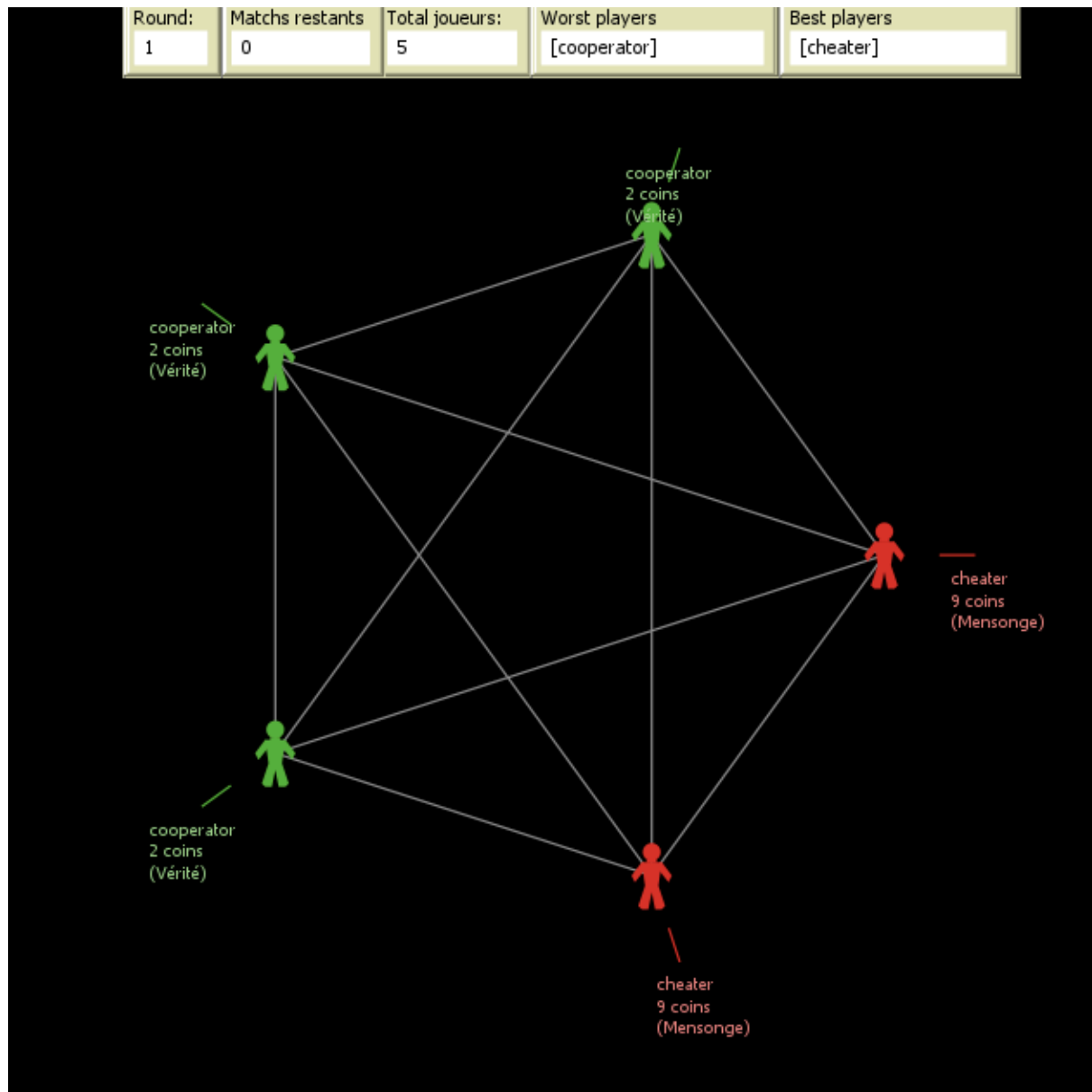


Image3: Cheater vs 4 Cooperators Tour 2

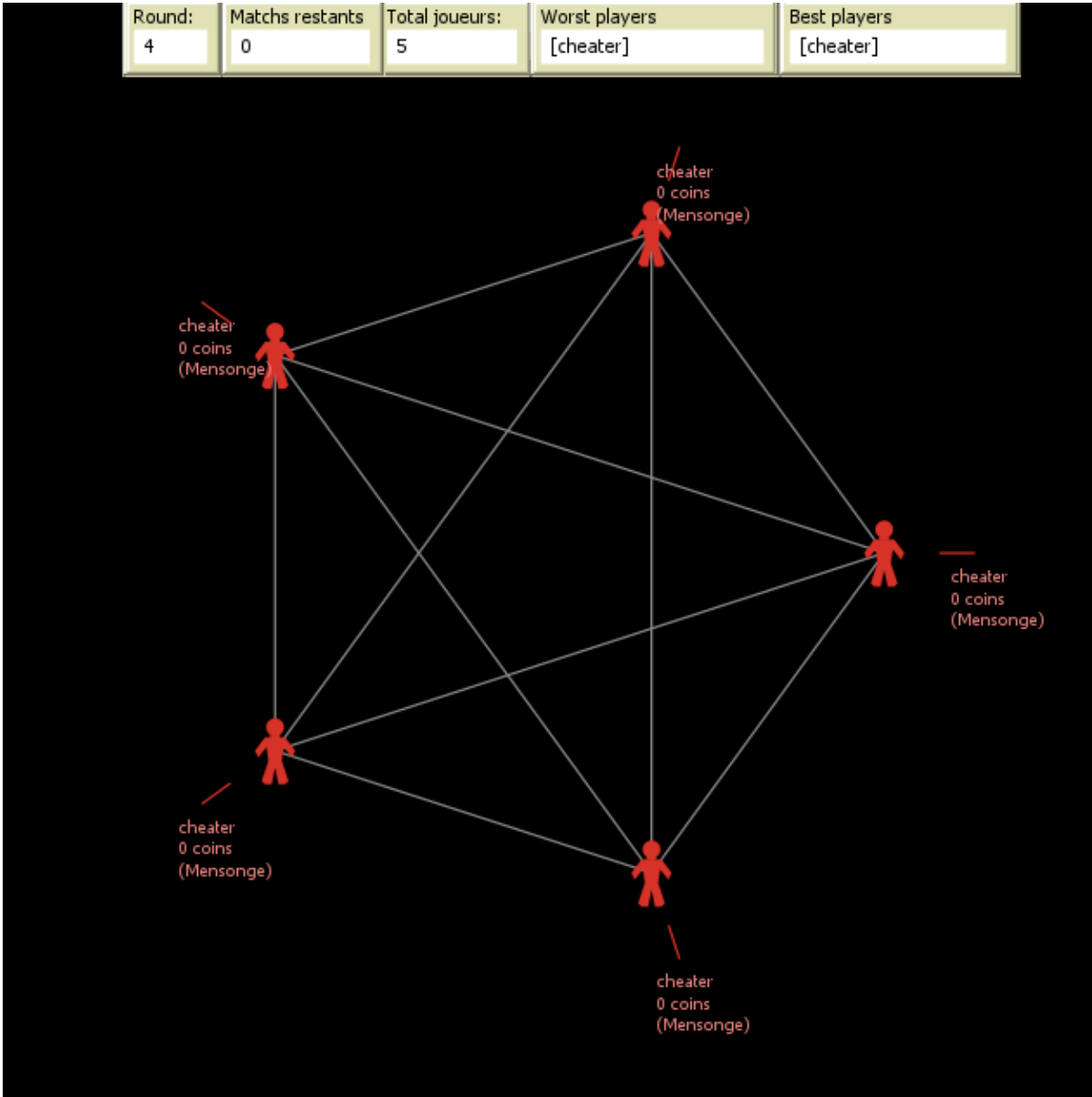


Image4: Cheater vs 4 Copycats tour 1

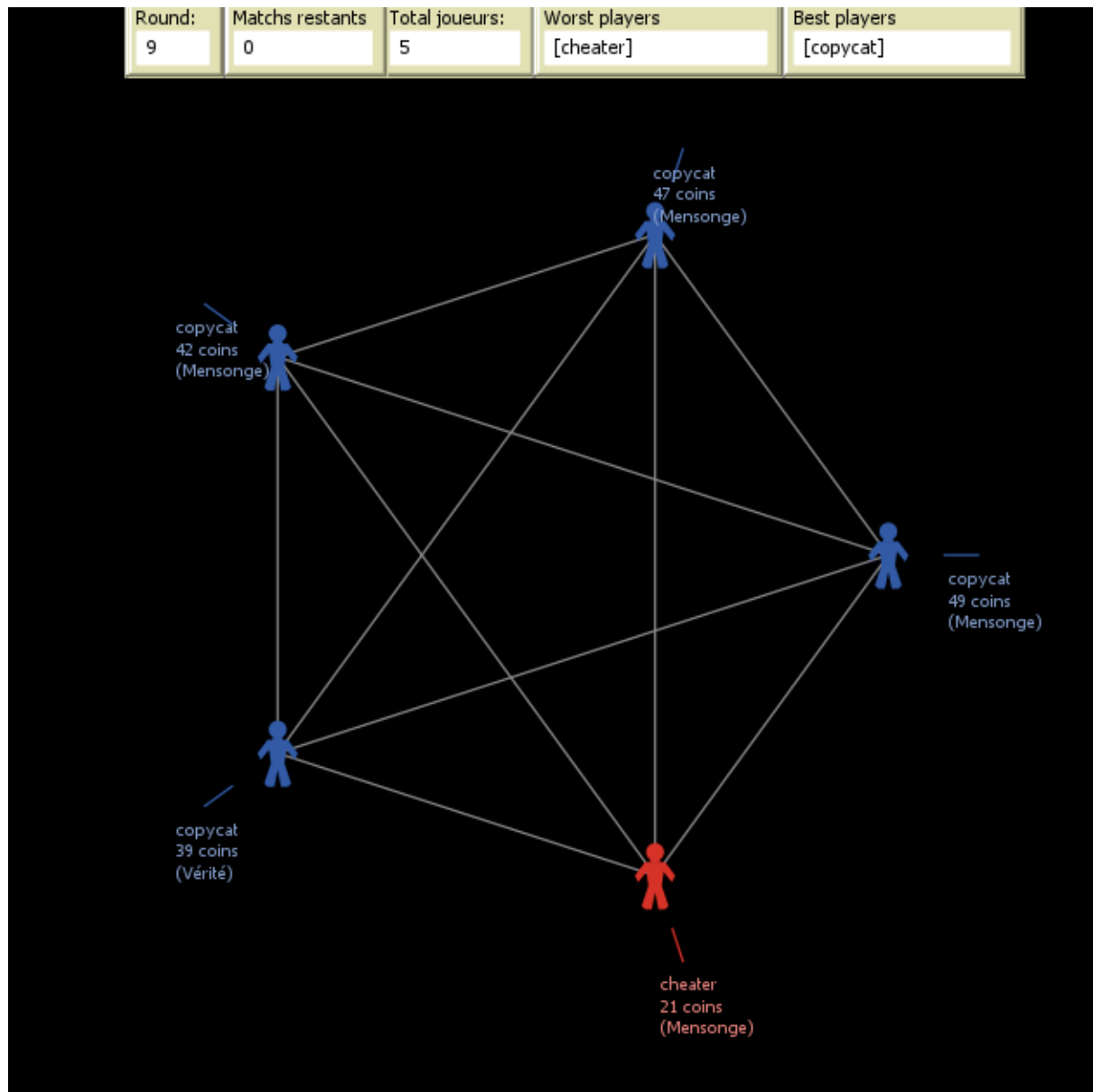


image5: Premier tour du match tous contre tous

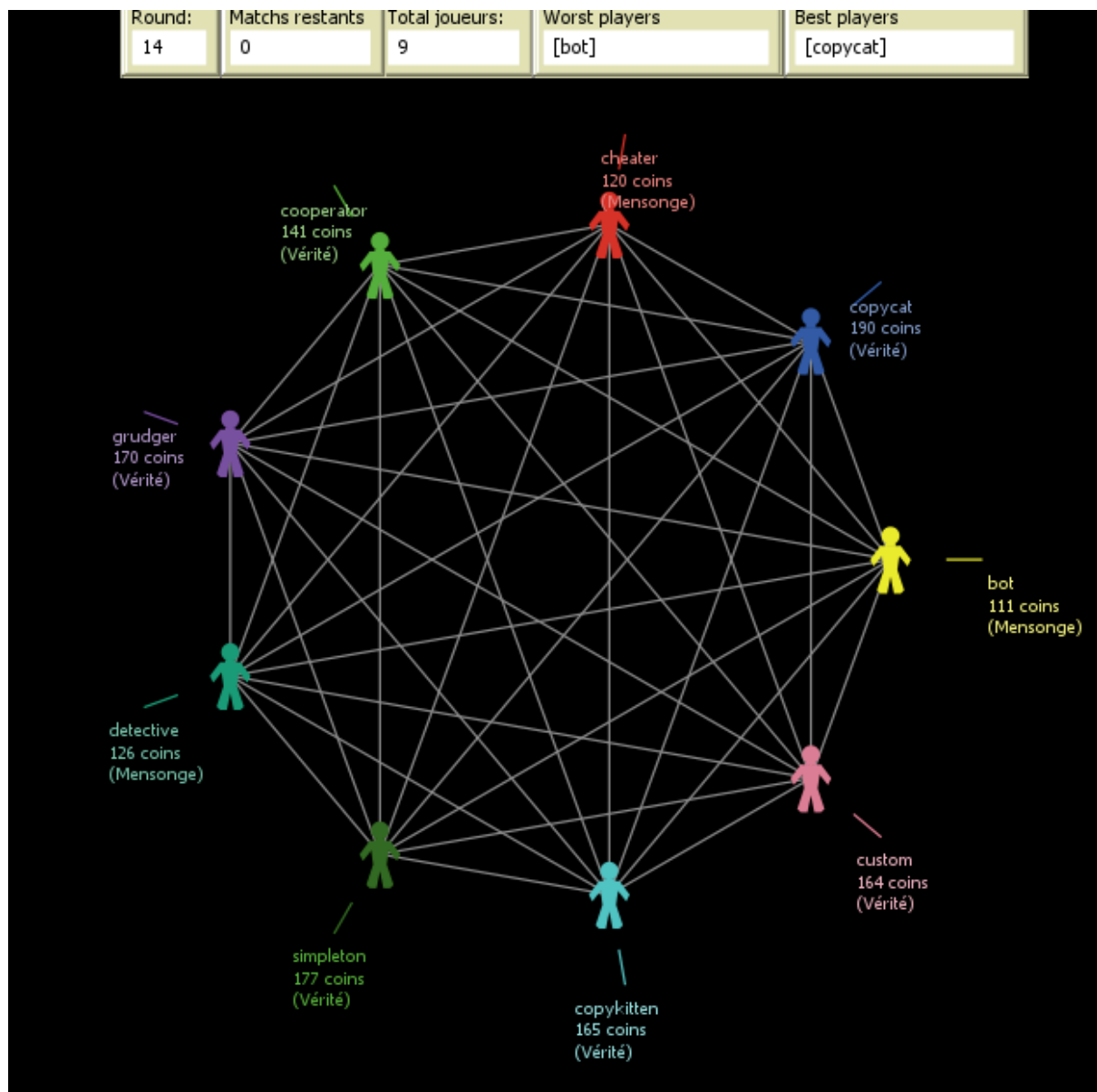


image6: Second tour du match tous contre tous

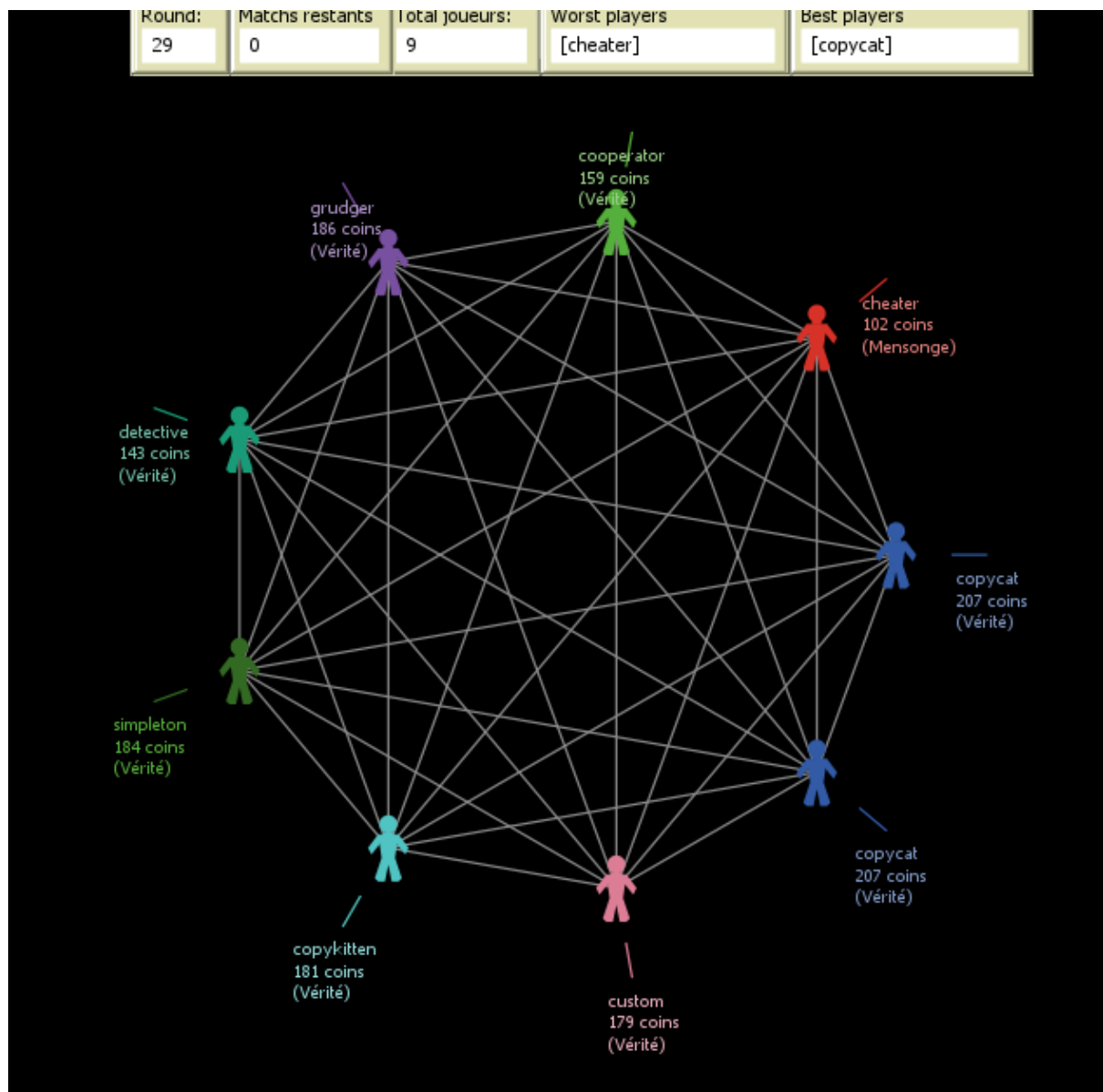


image7: Troisième tour du match tous contre tous

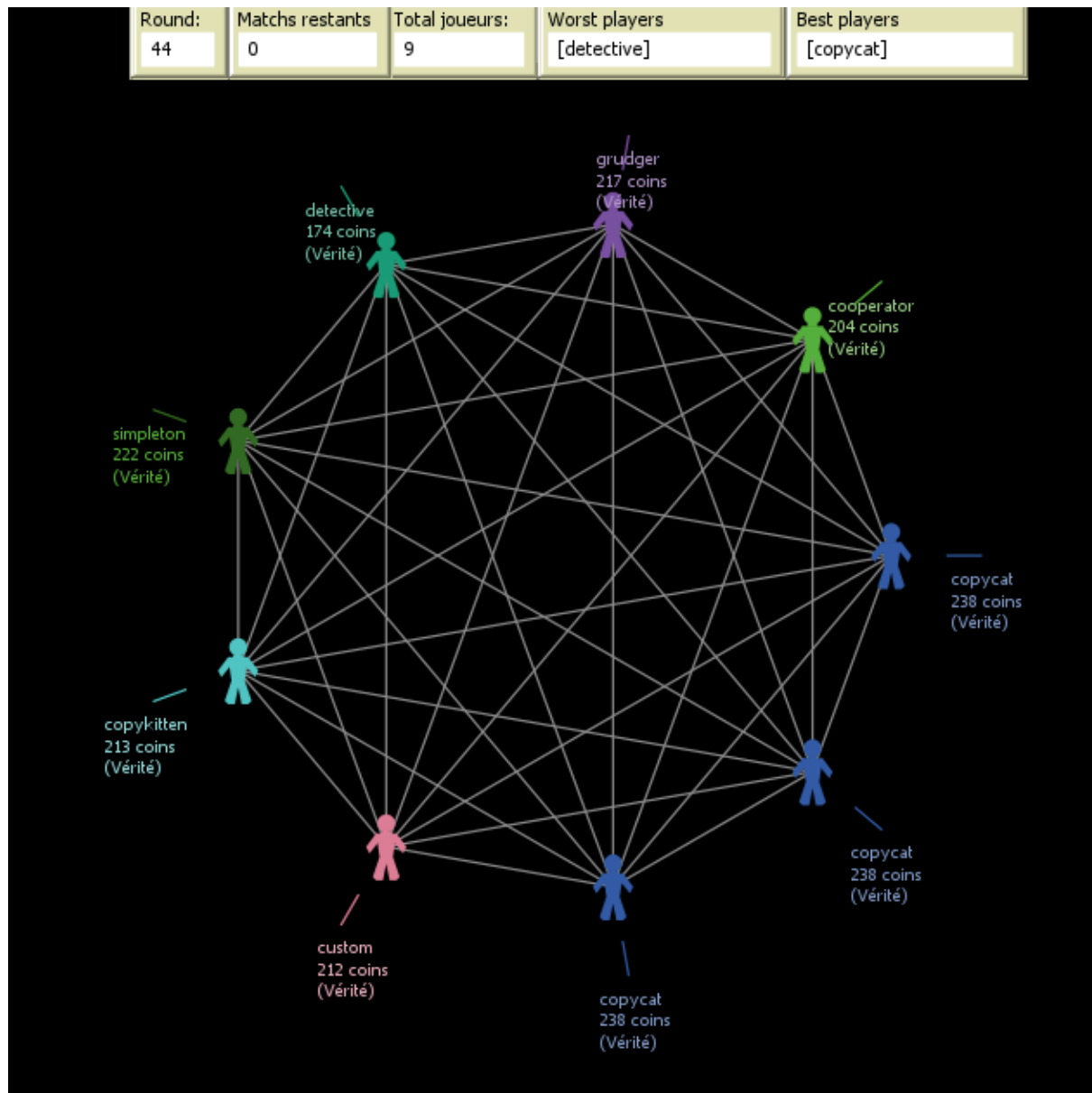


image8: Quatrième et dernier tour utile du match tous contre tous

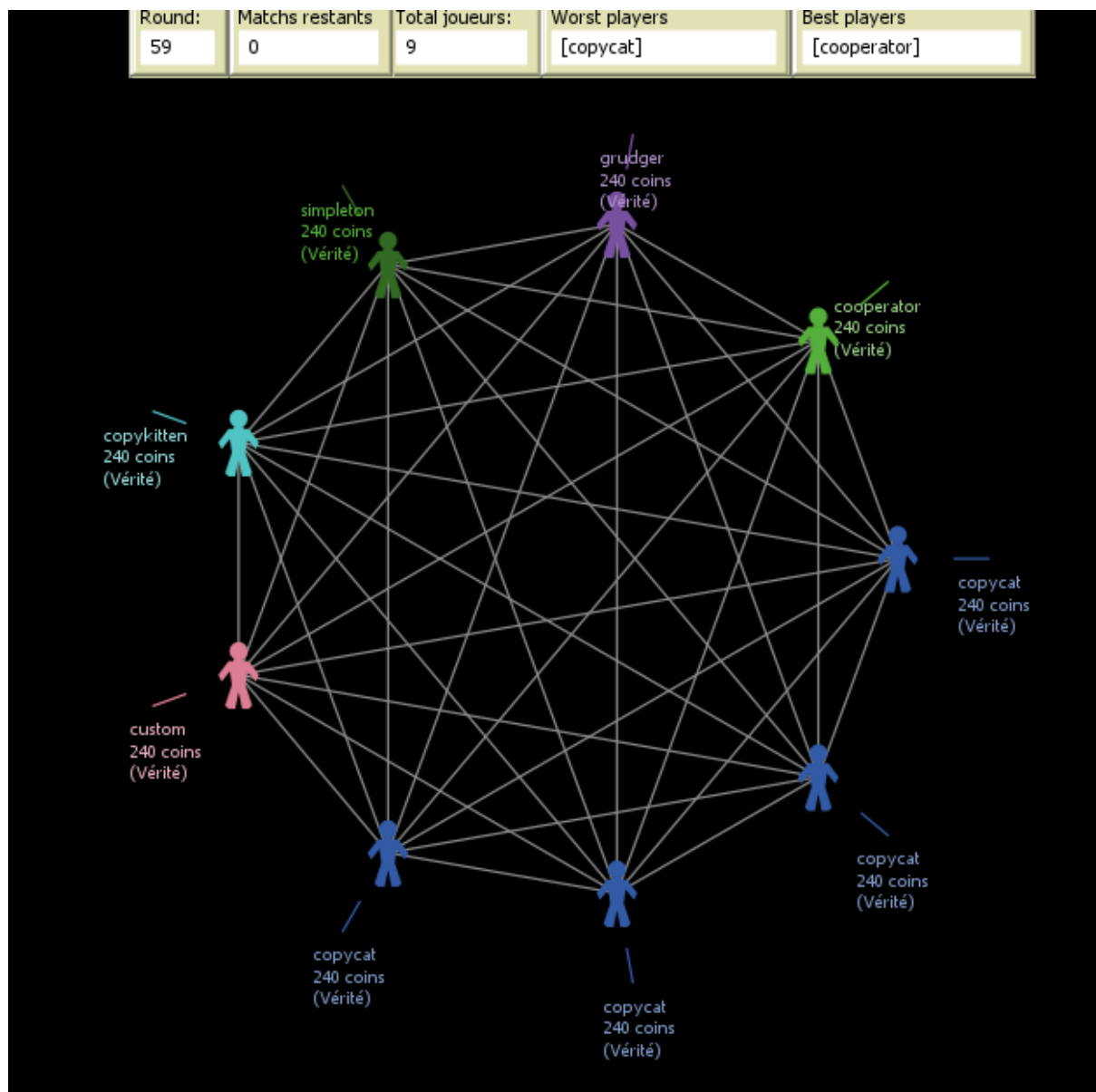


image9: Placement incorrect des noms par défaut

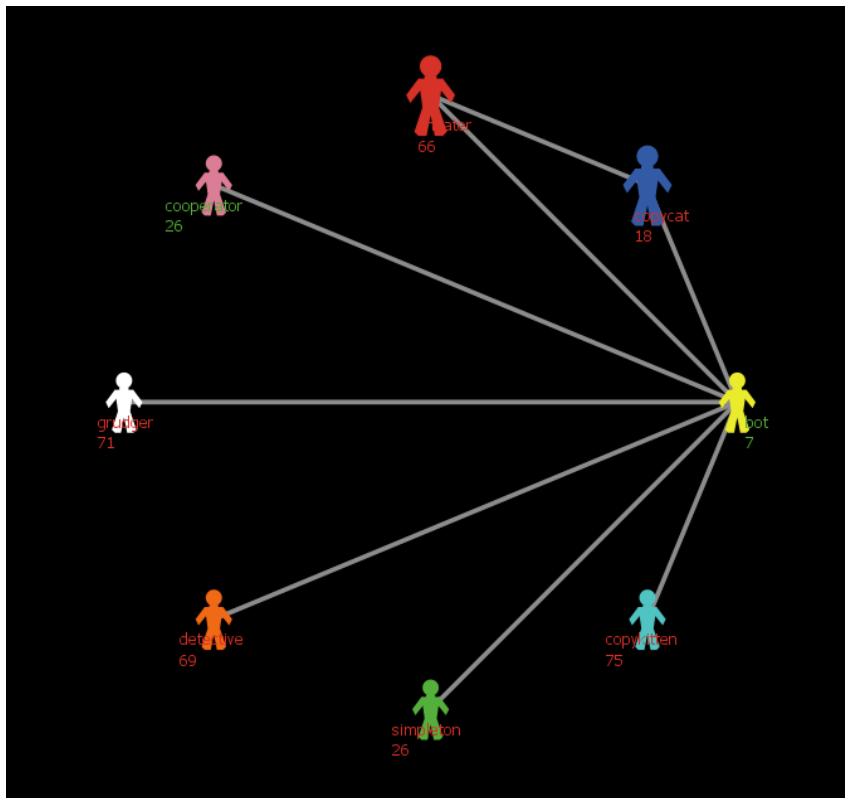


image10: Size de l'étiquette nul (placement des étiquettes sur les joueurs à droite)

